HW: LLMs, vectors, RAG :)

Summary

In this HW, you will:

- use Weaviate [https://weaviate.io/], which is a vector DB - stores data as vectors after vectorizing, and computes a search query by vectorizing it and does similarity search with existing vectors
- use lightning.ai [https://lightning.ai] to run RAG on lightning's 'GPU-accelerated cloud VM' platform
- use HuggingFace [https://huggingface.co/] to run a RAG application and describe its components

These are cutting-edge techniques to know, from a future/career POV :) Plus, they are simply, great FUN!!

Q1. (4 points)

Description

We are going to use vector-based similarity search, to retrieve search results that are not keyword-driven.

The (three) steps we need are really simple:

- install Weaviate plus vectorizer via Docker as images, run them as containers
- specify a schema for data, upload data/knowledge (in .json format) to have it be vectorized
- run a query (which also gets vectorized and then sim-searched), get back results (as JSON)

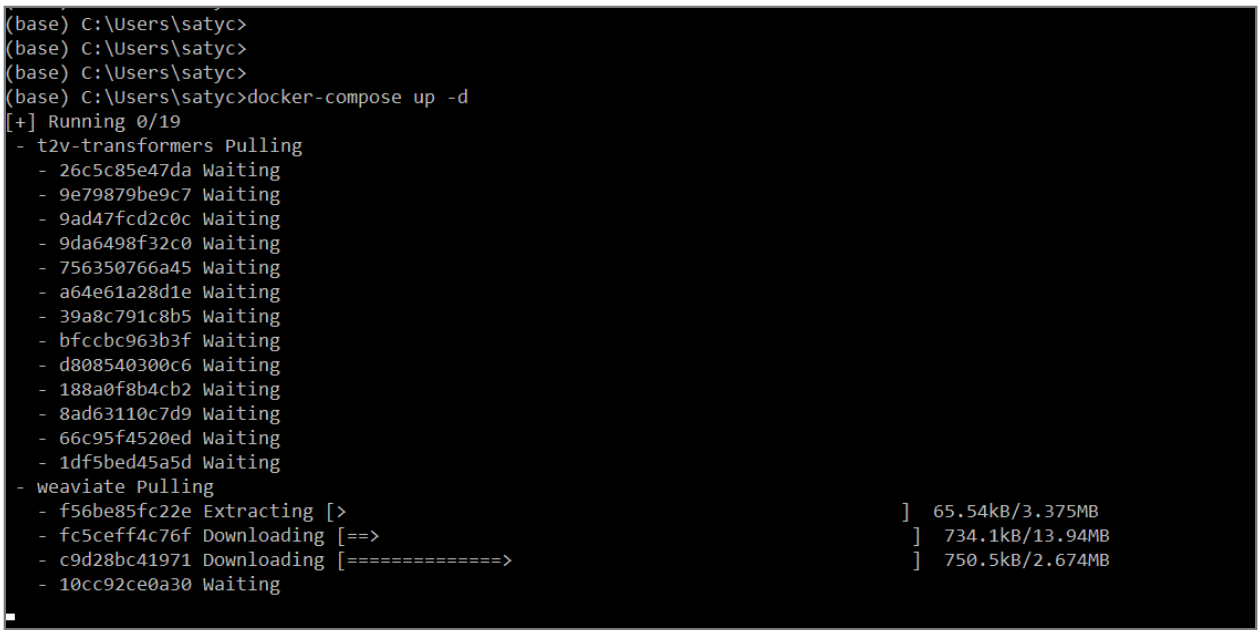
The following sections describe the above steps.

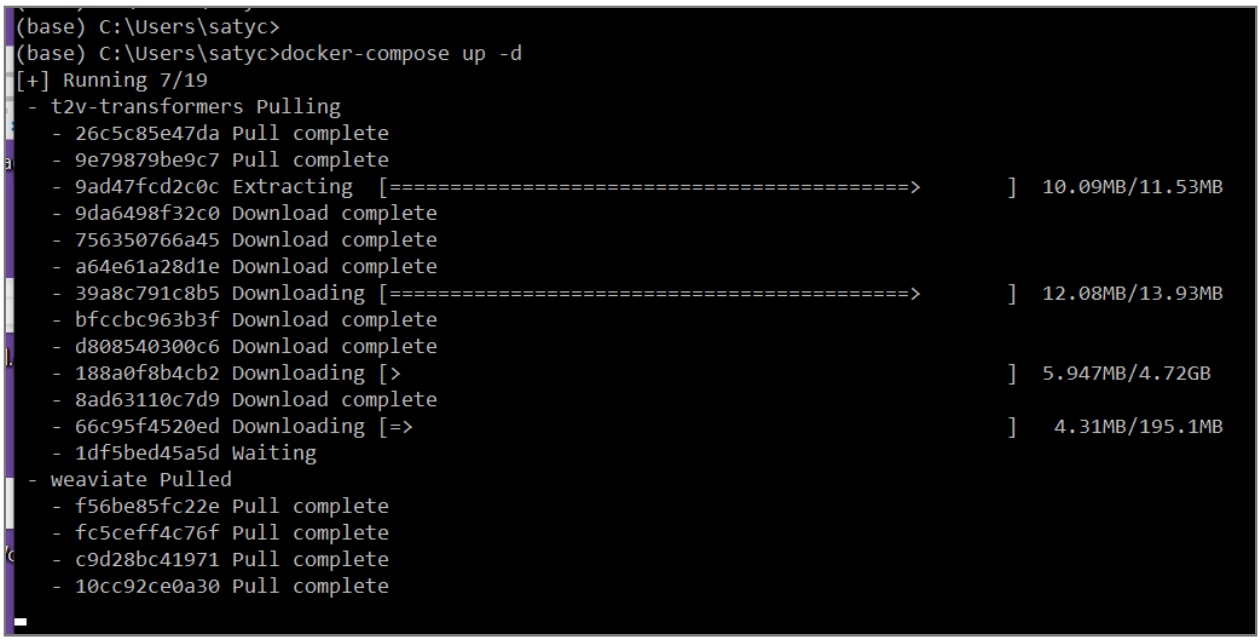
I. Installing Weaviate and a vectorizer module

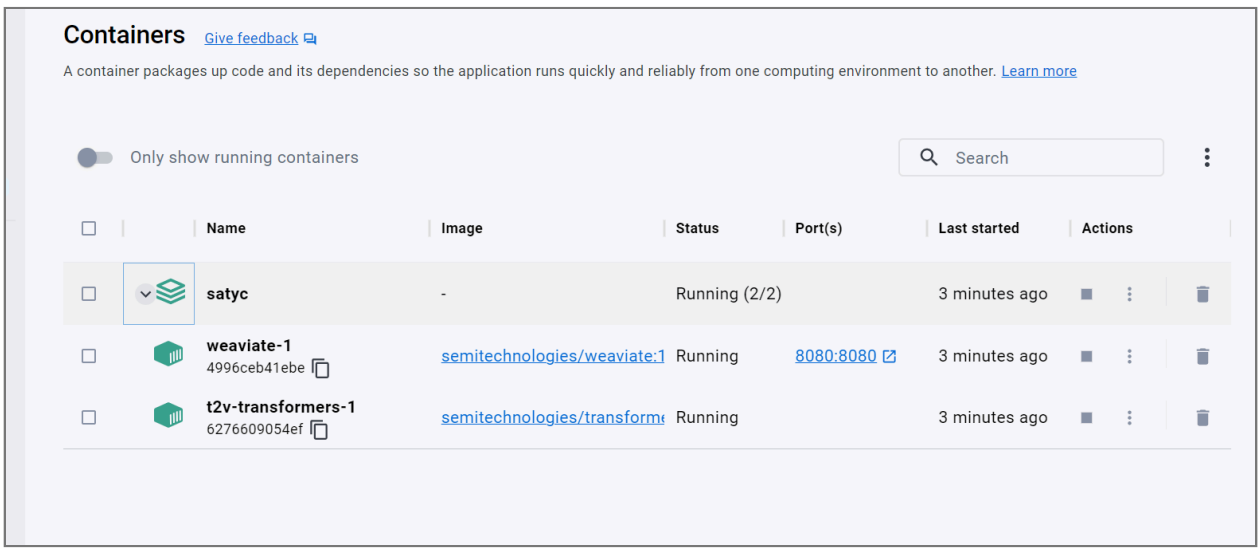
After installing Docker, bring it up (eg. on Windows, run Docker Desktop). Then, in your (ana)conda shell, run this docker-compose command that uses this yaml 'docker-compose.yml' config file to pull in two images: the 'weaviate' one, and a text2vec transformer called 't2v-transformers':

```
docker-compose up -d
```

These screenshots show the progress, completion, and subsequently, two containers automatically being started (one for weaviate, one for t2v-transformers):







Yeay! Now we have the vectorizer transformer (to convert sentences to vectors), and weaviate (our vector DB search engine) running! On to data handling :)

2. Loading data to search for

This is the data (knowledge, aka external memory, ie. prompt augmentation source) that we'd like searched, part of which will get returned to us as results. The data is represented as an array of JSON documents. Here is our data file, conveniently named data.json (you can rename it if you like) [you can visualize it better using https://jsoncrack.com] - place it in the 'root' directory of your webserver (see below). As you can see, each datum/'row'/JSON contains three k:v pairs, with 'Category', 'Question', 'Answer' as keys - as you might guess, it seems to be in Jeopardy(TM) answer-question (reversed) format :) The file is actually called jeopardy-tiny.json, I simply made a local copy called data.json.

The overall idea is this: we'd get the 10 documents vectorized, then specify a query word, eg. 'biology', and automagically have that pull up related docs, eg. the 'DNA' one (even if the search result doesn't contain 'biology' in it)! This is a really useful **semantic** search feature where we don't need to specify exact keywords to search for.

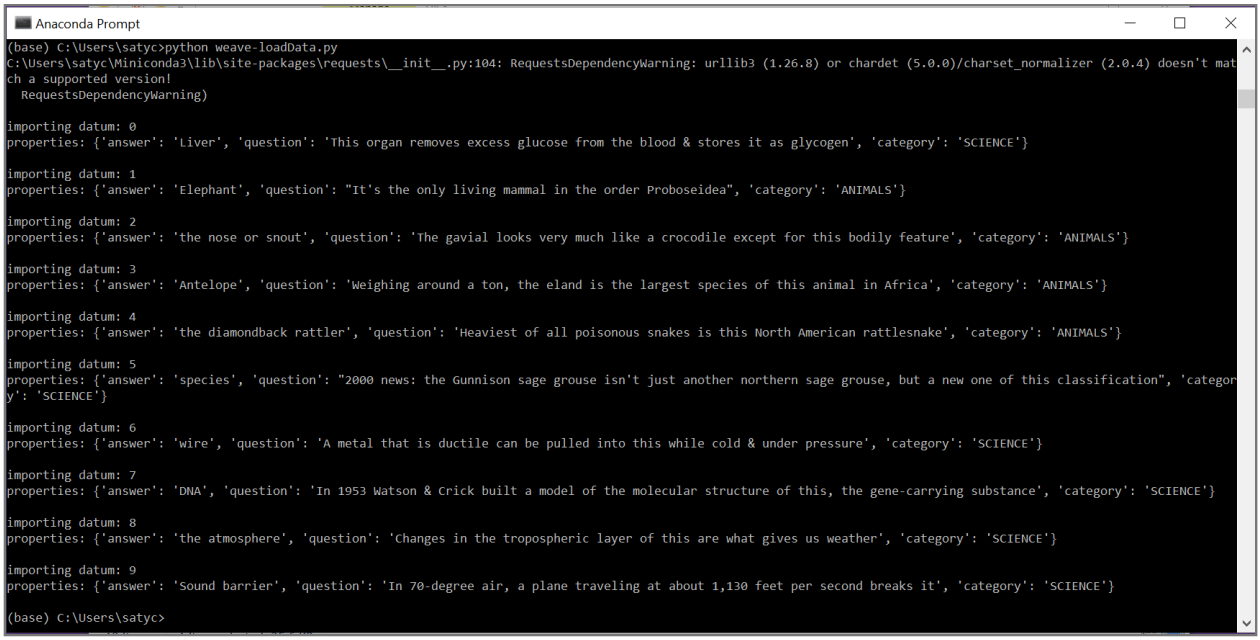
Start by installing the weaviate Python client:

```
pip install weaviate-client
```

So, how to submit our JSON data, to get it vectorized? Simply use this Python script, do:

```
python weave-loadData.py
```

You will see this:



If you look in the script, you'll see that we are creating a schema - we create a class called 'SimSearch' (you can call it something else if you like). The data we load into the DB, will be associated with this class (the last line in the script does this via add_data_object()).

NOTE - **you NEED to run a local webserver** [in a separate ana/conda (or other) shell], eg. via python 'serveit.py' - it's what will 'serve' data.json to weaviate :)

Great! Now we have specified our searchable data, which has been first vectorized (by 't2v-transformers'), then stored as vectors (in weaviate).

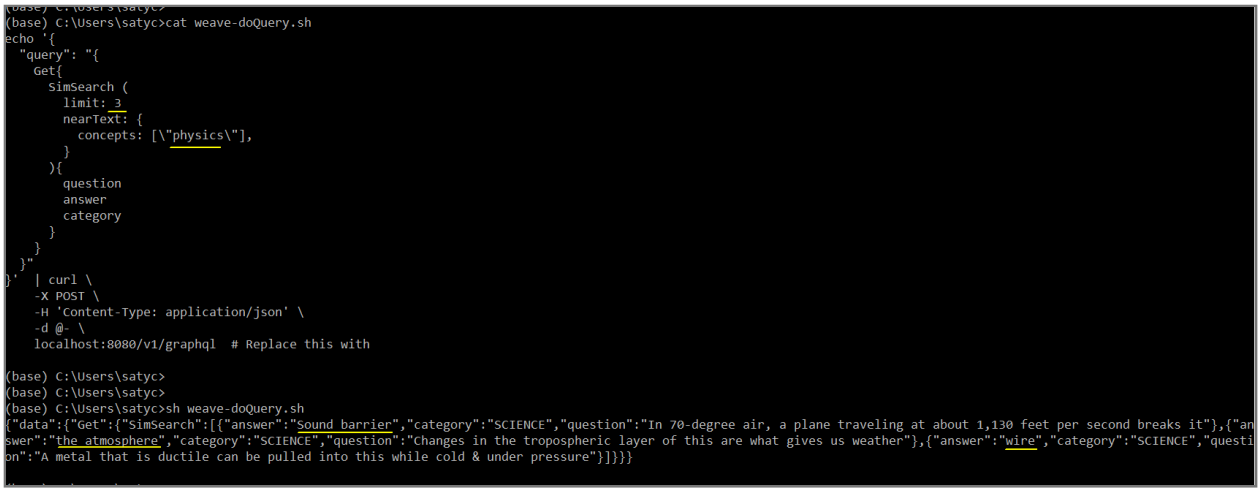
Only one thing left: querying!

3. Querying our vectorized data

To query, use this simple shell script called weave-doQuery.sh, and run this:

```
sh weave-doQuery.sh
```

As you can see in the script, we search for 'physics'-related docs, and sure enough, that's what we get:



Why is this exciting? Because the word 'physics' isn't in any of our results!

Now it's your turn:

• first, MODIFY the contents of data.json, to replace the 10 docs in it, with your own data, where you'd replace ("Category","Question","Answer") with ANYTHING you like, eg. ("Author","Book","Summary"), ("MusicGenre","SongTitle","Artist"), ("School","CourseName","CourseDesc"), etc, etc - HAVE fun coming up with this! You can certainly add more docs, eg. have 20 of them instead of 10

• next, MODIFY the query keyword(s) in the query .sh file - eg. you can query for 'computer science' courses, 'female' singer, 'American' books, ['Indian','Chinese'] food dishes (the query list can contain multiple items), etc. Like in the above screenshot, 'cat' the query, then run it, and get a screenshot to submit. BE SURE to also modify the data loader .py script, to put in your keys (instead of ("Category","Question","Answer"))

That's it, you're done :) In RL you will have a .json or .csv file (or data in other formats) with BILLIONS of items! Later, do feel free to play with bigger JSON files, eg. this 200K Jeopardy JSON file :)

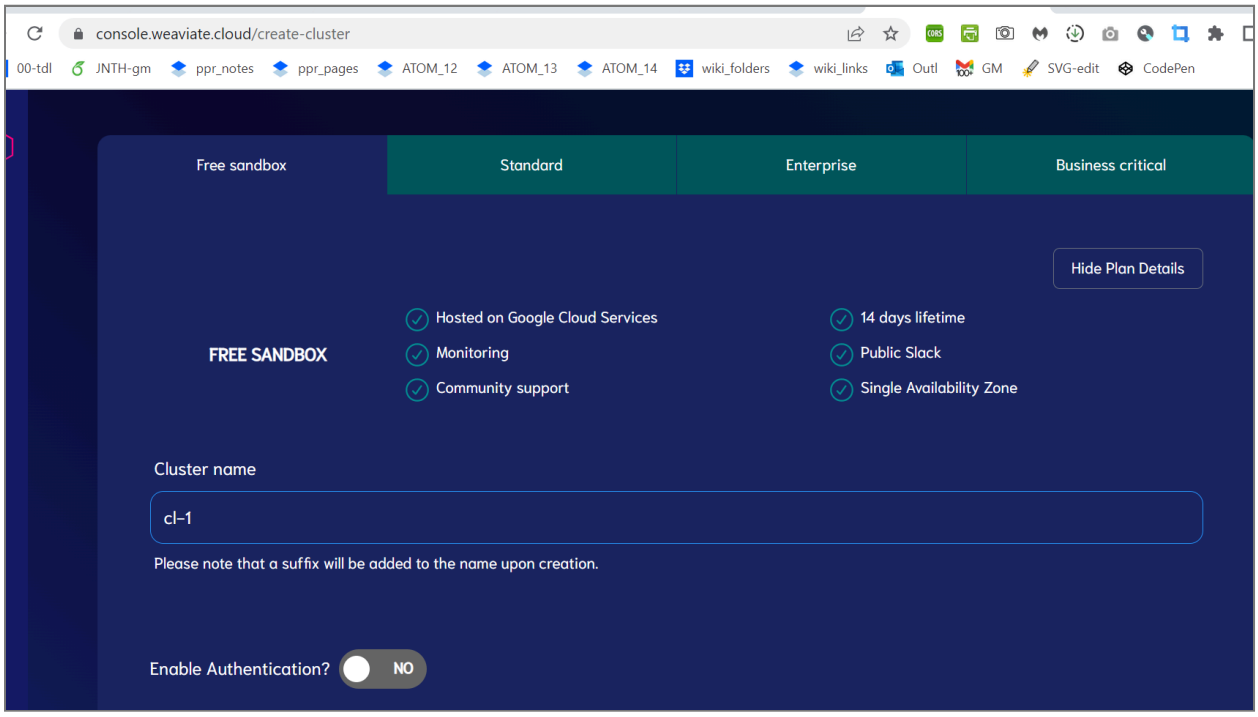
FYI/'extras' [for you to pursue later]

Here are two more things you can do, via 'curl':

[you can also do 'http://localhost:8080/v1/meta' in your browser]

[you can also do 'http://localhost:8080/v1/schema' in your browser]

Weaviate has a cloud version too, called WCS - you can try that as an alternative to using the Dockerized version:



Run this :)

Also, for fun, see if you can print the raw vectors for the data (the IO docs)...

More info:

- **https://weaviate.io/developers/weaviate/quickstart/end-to-end**
- **https://weaviate.io/developers/weaviate/installation/docker-compose**
- **https://medium.com/semi-technologies/what-weaviate-users-should-know-about-docker-containers-160lc6afa079**
- **https://weaviate.io/developers/weaviate/modules/retriever-vectorizer-modules/text2vec-transformers**

Q2. (3 points)

This is a quick, easy and useful one!

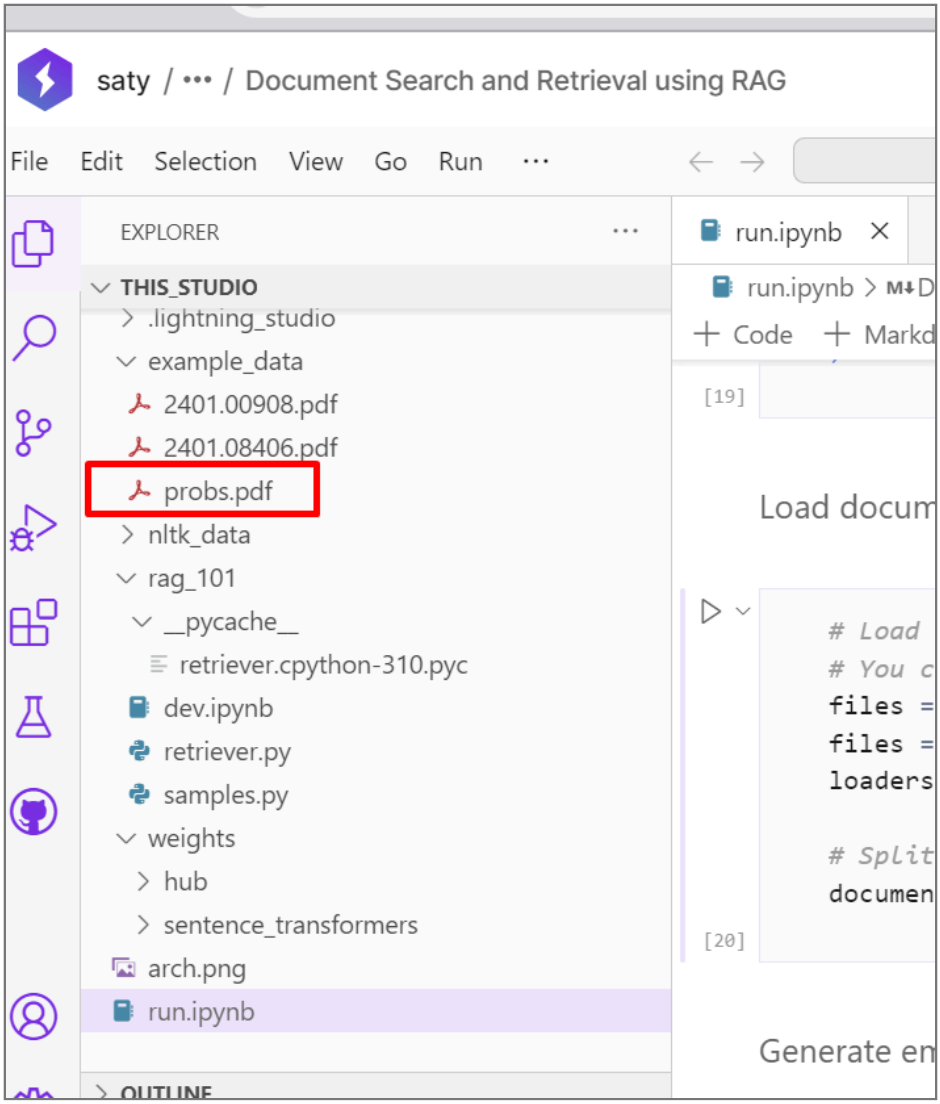
Go to https://lightning.ai/ and sign up for a free account. Then read these: https://lightning.ai/docs/overview/getting-started and https://lightning.ai/docs/overview/getting-started/studios-in-10-minutes

Browse through their vast collection of 'Studio' templates: https://lightning.ai/studios - when you create (instantiate) one, you get your own sandboxed environment [a 'cloud supercomputer'] that runs on lightning.ai's servers. You get unlimited CPU use, and 22 hours of GPU use per month (PLENTY, for beginner projects).

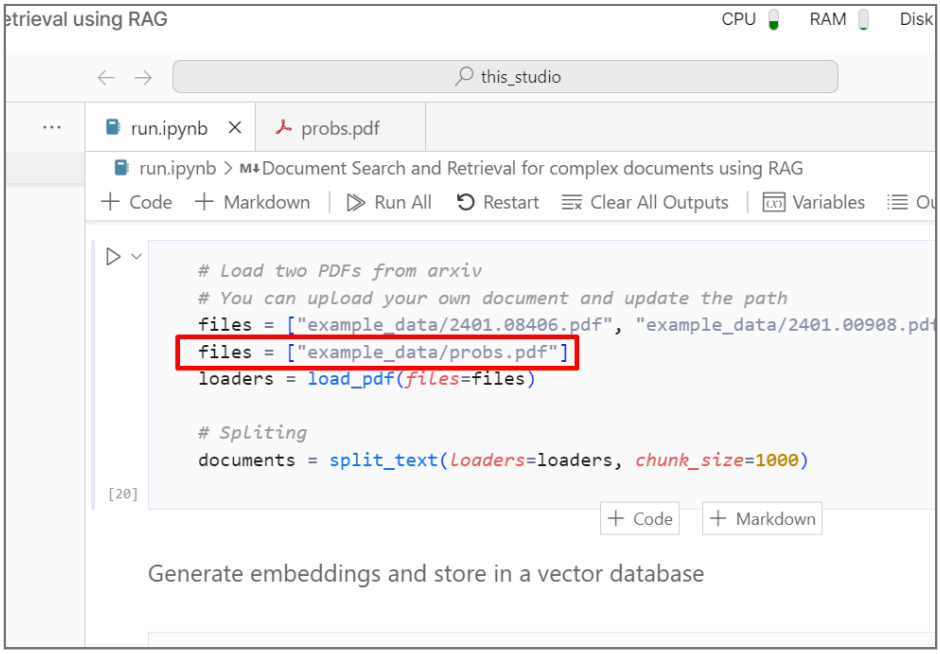
Create this Studio [ie click on it so the IDE comes up]: https://lightning.ai/lightning-ai/studios/document-search-and-retrieval-using-rag - you are going to use this to do RAG using your own PDF :)

Upload (drag and drop) a PDF [can be on ANY topic - coding, cooking, crafting, canoeing, cattle-ranching, catfishing... (lol!)]. Eg.

this shows the pdf I uploaded:

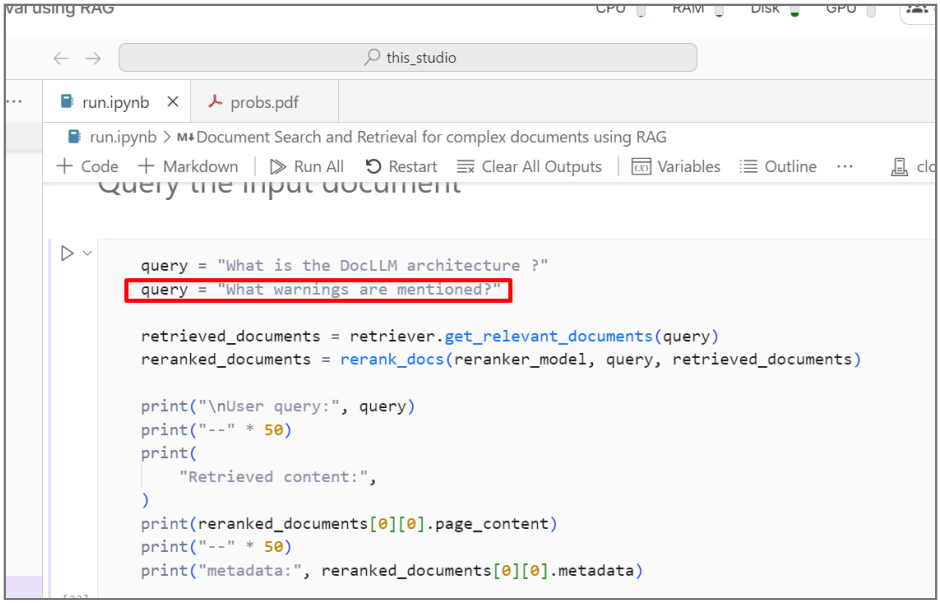


Next, edit run.ipynb, modify the 'files' variable to point to your pdf:



Modify the 'query' var, and the 'queries' var, to each contain a QUESTION on which you'd like to do RAG, ie. get the answers from the pdf you uploaded! THIS is the cool part - to be able to ask questions in natural language, rather than search by keyword, or

look up words in the index [if the pdf has an index].



Read through the notebook to understand what the code does and what the RAG architecture is, then run the code! You'll see the two answers printed out. Submit screenshots of your pdf file upload, the two questions, and the two answers. The answers might not be what you expected (ie might be imprecise, EVEN though it's RAG!) but that's ok - there are techniques to improve the quality of the retrieval, you can dive into them later.

After the course, DO make sure to run LOTS of templates! It's painless (zero installation!), fast (GPU execution!) and useful/informative (well-documented!). It doesn't get more cutting edge than these, for 'IR'. You can even write and run your own code in a Studio, and publish it as a template for others to use :)

Q3. (3 points)

This is also an easy+fun one...

Go to https://huggingface.co/spaces, scroll through the TONS of apps there, and pick a 'RAG' one [https://huggingface.co/spaces?sort=trending&search=RAG]. Run it, by uploading what's required (usually a PDF file).

Get a screenshot of your interacting with the app [similar to Q2].

Additionally, go through the source for the app [look in the 'Files' tab on the top right (sometimes it's in the ... icon)] and:

• describe the RAG steps involved (ie. the Python calls) - you don't need to do this in exhaustive detail, but you DO need to describe the workflow (that starts with ingesting the input, ending with output generation)

• describe how the UI works (what calls do whatwhat) - the file component, input, output. The UI is generated via Gradio [https://www.gradio.app/] or Streamlit [https://streamlit.io/], so this exercise is to get you familiar with their basics :)

Simply submit screenshots (for QI, Q2, Q3) and a text file (for Q3), all as a single .zip file.

Getting help

There is a hw4 'forum' on Piazza, for you to post questions/answers. You can also meet w/ the TAs, CPs, or me.

Have fun! These (LLMs, vectorizing, RAG, Python-based UI spec...) are really (REALLY!!!) useful pieces of 'tech' to know. In the coming years, LLMs are sure to make their way into EVERY app/site/backend/system/process.